# Solving the Product Breakdown Structure Problem with constrained QAOA

René Zander[1], Raphael Seidel[1], Matteo Inajetovic[2], Niklas Steinmann[1], and Matic Petrič[1]

[1]Fraunhofer Institute for Open Communication Systems, Berlin, Germany
[2]Technische Universität Berlin, Berlin, Germany

arXiv:2406.15228v1 [quant-ph] 21 Jun 2024

**Constrained optimization problems, where not all possible variable assignments are feasible solutions, comprise numerous practically relevant optimization problems such as the Traveling Salesman Problem (TSP), or portfolio optimization. Established methods such as quantum annealing or vanilla QAOA usually transform the problem statement into a QUBO (Quadratic Unconstrained Binary Optimization) form, where the constraints are enforced by auxiliary terms in the QUBO objective. Consequently, such approaches fail to utilize the additional structure provided by the constraints.**

**In this paper, we present a method for solving the industry relevant Product Breakdown Structure problem. Our solution is based on constrained QAOA, which by construction never explores the part of the Hilbert space that represents solutions forbidden by the problem constraints. The size of the search space is thereby reduced significantly. We experimentally show that this approach has not only a very favorable scaling behavior, but also appears to suppress the negative effects of Barren Plateaus.**

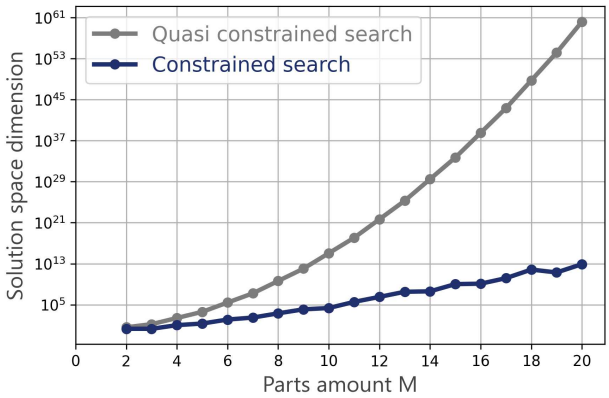



Figure 1: Comparison of the state space dimensions of the constrained search space versus the quasi constrained search. Quasi constrained means that the full space is admitted but the problem constraints are enforced by encoding them into the cost function. The PBS graph instances, encoding the constraints, to evaluate the data are randomly generated tree graphs with maximum degree $N = \lfloor M/2 \rfloor$. For each $M$ we generate 30 such graphs and evaluate the average. Note that at $M = 20$ the solution space of the quasi constrained approach differs by 46 orders of magnitude, demonstrating a very clear scaling advantage.

## 1 Introduction

In this paper, we develop a novel approach for solving a combinatorial optimization problem with high relevance in logistics and supply chain management, namely the product breakdown structure (PBS) problem [1, 2]. A key challenge in supply chain optimization is determining how to best distribute parts manufacturing or assembly amongst possible suppliers at different geographical locations in order to minimize carbon dioxide emissions and ensure a reliable and efficient supply chain for manufacturing processes. The application of quantum solutions in logistics promises to accelerate the path towards sustainable and more efficient supply chains.

René Zander: rene.zander@fokus.fraunhofer.de
Raphael Seidel: raphael.seidel@fokus.fraunhofer.de
Matteo Inajetovic: m.inajetovic@tu-berlin.de
Niklas Steinmann: niklas.steinmann@fokus.fraunhofer.de
Matic Petrič: matic.petric@fokus.fraunhofer.de

The PBS problem belongs to the class of constrained optimization problems. In this context, *constrained* means that not every variable assignment results in a feasible solution. This class of problems comprises a wide array of practically relevant optimization problems. While widely-used methods, such as quantum annealing or the plain vanilla Quantum Approximate Optimization Algorithm (QAOA), usually transform the problem statement into a Quadratic Unconstrained Binary Optimization (QUBO) form, we purposefully avoid this step. This is because with the conversion to a QUBO problem statement, said problem structure is usually irreversibly destroyed and can no longer be leveraged.

In this work, we utilize a modified QAOA approach, which by construction never reaches the part of the Hilbert space that represents solutions forbidden by the constraints. This, by itself, already yields powerful advantages as a result of the size of the search

space being significantly reduced (see Fig. 1). This is achieved by designing a constraint-specific state preparation algorithm which creates a superposition of all feasible solutions. Thereby, a constrained mixer that leaves the subspace of feasible solutions invariant is constructed.

The outlined method is implemented in the high-level programming framework *Eclipse Qrisp* [3–5]. *Qrisp* enables developers to write quantum code with ease, facilitating a systematic approach to quantum software engineering. Therewith, quantum computing is shifted from the domain of research and experiments to the domain of programmers, applications and businesses.

The paper is structured as follows: We start by defining the PBS problem in Section 2, before providing a short overview over the Quantum Approximate Optimization Algorithm [6] together with some of its variants addressing problem constraints [7, 8] in Section 3. The concept of constrained mixers is elaborated on in Section 4. In Section 5, we introduce a novel constraint-specific state preparation algorithm. Subsequently, we present compilation techniques for the constrained mixer that are specifically tailored to the used state preparation algorithm. This step is taken to ensure that the required quantum resources are reduced to a minimal amount. In Section 6, we cover the implementation of our method in *Qrisp*, and benchmark its performance and solution quality. A further scaling advantage is verified by a small study of our approach regarding Barren Plateau susceptibility in Appendix B. Finally, in Section 7, we propose several interesting directions for further improvement of our approach in future research.

## 2 Product breakdown structure problem

A product breakdown structure in manufacturing can be defined by a rooted directed tree[1] $T$ with set of nodes $[M] = \{1, \dots, M\}$. The nodes correspond to parts with the root being the final product.

A directed edge is given by a tuple $(r, s)$ for $r, s \in [M]$ indicating that the part $r$ is a sub-part of $s$, e.g., $r$ is a predecessor of $s$. Let $\phi$ denote the set of (directed) edges. The set of predecessors of a node is denoted by $\mathrm{pred}(x)$. Let $d_x = |\mathrm{pred}(x)|$ be the degree of the node $x$, and let $m = \max_{x \in T} d_x$ the maximum degree of all nodes.

The product breakdown structure problem [1,2] can be described as follows: For a set $[N] = \{1, \dots, N\}$ of sites, find an assignment

$$f \colon [M] \to [N] \tag{1}$$

[1]Here, the directed edges point from the leaves to the root.

that satisfies the constraints

$$f(r) \neq f(s), \quad \text{for all } (r, s) \in \{x\} \cup \mathrm{pred}(x), \tag{2}$$

for all nodes $x \in T$ and minimizes the cost function

$$C = \sum_{(r,s) \in \phi} c^r_{f(r), f(s)} \tag{3}$$

where $c^r_{i,j} > 0$ are real numbers. We assume that $c^r_{i,j} = c^r_{j,i}$ for all $r, i, j$. Here, $c^r_{i,j}$ represents the cost (or carbon emissions) of transporting part $r$ from site $i$ to site $j$.

With this, an instance of the PBS problem is characterized by a tuple $(T, \{c^x_{i,j}\}, N)$. Here, the problem constraints are encoded by the tree $T$.

The constraints (2) can be equivalently formulated as:

(i) Origin and destination of each part must be different.

(ii) The origins of any two sub-parts of a part must be different.

Clearly, if $N < m + 1$, no assignment satisfying the constraints (2) exists. We denote the set of feasible solutions by $\mathcal{F} \subset [N]^M$.

Define the binary variables $x_{r,i}$ for $r = 1, \dots, M$ and $i = 1, \dots, N$. Then the cost function can be expressed as:

$$C = \sum_{(r,s) \in \phi} \sum_{i \neq j} c^r_{i,j} x_{r,i} x_{s,j}. \tag{4}$$

The additional constraint

$$C_1 = \sum_{r \in [M]} \left( \sum_i x_{r,i} - 1 \right)^2 = 0 \tag{C1}$$

ensures that each part is assigned to exactly one site. The constraints (i) and (ii) can be expressed as:

$$C_2 = \sum_{(r,s) \in \phi} \sum_i x_{r,i} x_{s,i} = 0. \tag{C2}$$

and

$$C_3 = \sum_{(r,s) \in \psi} \sum_i x_{r,i} x_{s,i} = 0. \tag{C3}$$

where $\psi = \bigcup_x \{(r, s) \mid r, s \in \mathrm{pred}(x) \text{ and } r < s\}$ is the set of (ordered) pairs of sub-parts of any part. This combination of objective and constraints can be formulated as a quadratic unconstrained binary optimization (QUBO) through the addition of penalty terms to the objective with appropriate positive values for the coefficients $\lambda_1$, $\lambda_2$, and $\lambda_3$:

$$Q = C + \lambda_1 C_1 + \lambda_2 C_2 + \lambda_3 C_3. \tag{5}$$

## 3 Quantum Approximate Optimization Algorithm

The Quantum Approximate Optimization Algorithm (QAOA) is a hybrid quantum-classical variational algorithm designed for solving combinatorial optimization problems [6]. The quantum and classical components work together iteratively: The quantum computer repeatedly prepares a parameter dependent quantum state and measures it, producing a classical output; this output is then fed into a classical optimization routine which produces new parameters for the quantum part. This process is repeated until the algorithm converges to an optimal or near-optimal solution.

As a first step, an optimization problem is formulated as

$$z = \text{argmin}_{x \in D} C(x), \tag{6}$$

where $C(x)\colon D \to \mathbb{R}$ is the cost function acting on $N$ variables $x = (x_1, \ldots, x_N)$, and $D \subset \mathbb{Z}^N$. Typical choices are $D = \{0,1\}^N$ or $D = \{0, \ldots, k\}^N$ (graph coloring with $k$ colors). The domain $D$ is encoded by computational basis states of a system of qubits with Hilbert space $\mathcal{H}$.

The QAOA operates in a sequence of layers, each consisting of a problem-specific operator and a mixing operator. To be precise, an initial state $|\psi_0\rangle$ is evolved under the action of $p$ layers of QAOA, where one layer consists of applying the unitary phase separating operator

$$U_P(C, \gamma) = e^{-i\gamma C},$$

which applies a phase to each computational basis state based on its cost function value; and the unitary mixer operator

$$U_M(B, \beta) = e^{-i\beta B},$$

where $B$ represents a specific mixer Hamiltonian that drives the transitions between different states. Depending on the unitaries' eigenvalues, the QAOA parameters are typically bound to hold values $\gamma \in \{0, 2\pi\}$, and $\beta \in \{0, \pi\}$, which are then optimized classically.

After $p$ layers of QAOA, we can define the angle dependent quantum state

$$|\psi_p\rangle = U_M(\beta_p) U_P(\gamma_p) \cdots U_M(\beta_1) U_P(\gamma_1) |\psi_0\rangle, \tag{7}$$

where $U_M(\beta) = U_M(B, \beta)$ and $U_P(\gamma) = U_P(C, \gamma)$.

The ultimate goal in QAOA is to optimize the variational parameters $\beta_1, \ldots, \beta_p$ and $\gamma_1, \ldots, \gamma_p$ in order to minimize the expectation value of the cost function $C$ with respect to the final state $|\psi_p\rangle$. This is achieved using classical optimization techniques.

For constrained optimization problems the set of feasible solutions $\mathcal{F} \subset \mathcal{H}$ is a strict subset of the Hilbert space $\mathcal{H}$. In this case, reducing the search space by exploring only solutions that satisfy the constraints of a given problem should yield better results. This is addressed in the scope of the Quantum Alternating Operator Ansatz [7]. By design, the initial state is trivial (constant depth) to implement, and the mixer is required to preserve the feasible subspace. Hadfield et. al. [7] provide an extensive list to such mixers for a variety of optimization problems including Max-$\kappa$-Colorable Subgraph or the Traveling Salesman Problem.

In this paper, we follow a slightly different strategy [8, 9]: We construct an algorithm for preparing a uniform superposition of all feasible states. That is, this algorithm implements a unitary $U_F$ such that $|\psi_F\rangle = U_F |0\rangle$ where

$$|\psi_F\rangle = \frac{1}{\sqrt{\mathcal{F}}} \sum_{x \in \mathcal{F}} |x\rangle .$$

Then the initial state for the QAOA is set to $|\psi_0\rangle = |\psi_F\rangle$. As explained in Section 4 below, given such a state preparation algorithm, a mixer that preserves the feasible subspace can be constructed.

Intuitively, the former approach starts in some isolated solution and transitions to the optimal solution through successive mixer applications, while the latter starts in a superposition of all feasible solutions and concentrates on the optimal solution through successive mixer applications. An interesting study would be a comparison of the two methods for different constrained optimization problems such as the TSP.

## 4 Constrained Mixers

A major hurdle of solving problems like the PBS problem using a QUBO solver are the constraints. Even though constraints can be encoded by augmenting the cost function by a term of the form $\lambda F(x)$ (where $F(x) = 1$ if $x$ is forbidden by the constraints and 0 otherwise), this approach brings several disadvantages with it that make it very impractical when it comes to solving actual problems:

- Compared to a classical algorithm, the quantum algorithm has a much larger search space because it also has to search through the forbidden states.
- It is not clear how to choose $\lambda$. This parameter needs to be big enough, such that forbidden states are effectively suppressed but small enough such that the hardware precision can still "resolve" the contrast in the actual cost function.
- Forbidden states have a non-zero probability of appearing as a solution, reducing the overall efficiency of the algorithm.

An interesting technique to overcome these problems is to encode the constraints into the mixer, such

that the algorithm never leaves the "allowed" space. This idea has been proposed in [8] and will be further refined within this work. Let

$$F : \mathbb{F}_2^n \to \mathbb{F}_2,\ x \to F(x) \tag{8}$$

represent an arbitrary constraint function. We say $x \in \mathbb{F}_2^n$ is allowed if $F(x) = 1$ and $x$ is forbidden otherwise.

We define the uniform superposition state

$$|\psi_F\rangle = \frac{1}{\sqrt{N_F}} \sum_{F(x)=1} |x\rangle \tag{9}$$

where $N_F = |\{x \mid F(x) = 1)\}|$.

Assume that there is a quantum circuit $U_F$ preparing $|\psi_F\rangle$ (a procedure for compiling these efficiently for the PBS problem will be presented in the next section):

$$U_F |0\rangle = |\psi_F\rangle \tag{10}$$

We now conjugate a multi-controlled phase gate controlled on the $|0\rangle$ state with $U_F$. This yields the following unitary:

$$\begin{aligned} U_M(\beta) &= U_F \mathrm{MCP}_0(\beta) U_F^\dagger \\ &= U_F(1 - (1 - \exp(i\beta)) |0\rangle \langle 0|) U_F^\dagger \\ &= 1 - (1 - \exp(i\beta))(U_F |0\rangle)(U_F |0\rangle)^\dagger \\ &= 1 - (1 - \exp(i\beta)) |\psi_F\rangle \langle \psi_F| \end{aligned} \tag{11}$$

This quantum circuit satisfies the following properties, which classify it as a valid constrained QAOA mixer

- $U_M(\beta) |x\rangle = |x\rangle$ if $F(x) = 0$ (follows directly from $\langle \psi_F | x\rangle = 0$). This property makes sure that forbidden states are mapped onto themselves, guaranteeing that the mixer only mixes among the allowed states.
- $U_M(0) = 1$. This property ensures that there is indeed no mixing happening at $\beta = 0$.
- $|\langle x| U_M(\beta) |x\rangle| \neq 1$ for $F(x) = 1, \beta \in (0, 2\pi)$. This property shows that there is indeed **some** mixing happening for allowed states at $\beta \neq 0$.

### 4.1 Benefits of constrained mixers

As it turns out, the constrained mixer approach improves the optimization process significantly. Firstly, since the state space dimension is heavily reduced (compared to encoding the constraints in the cost function), the algorithm has to search through a much smaller solution space. This naturally improves the efficiency and scalability. Furthermore, the obtained state space reduction also mitigates the well-known Barren Plateau phenomenon [10], which depends on the Hilbert space dimension and negatively affects a lot of Variational Quantum Algorithms. Consider a generic parameterized quantum circuit:

$$U(\boldsymbol{\theta}) = \prod_{k=1}^{L} U_k(\boldsymbol{\theta}_k)$$

acting on a Hilbert space $\mathcal{H} = (\mathbb{C}^2)^{\otimes n}$ and a cost function $C(\boldsymbol{\theta}) = Tr[OU(\boldsymbol{\theta})\rho U^\dagger(\boldsymbol{\theta})]$, where $O$ is a Hermitian observable, $\boldsymbol{\theta}$ the set of parameters, $L$ the number of layers, and $\rho$ the initial state. Barren plateaus manifest when $\mathrm{Var}_\theta(\partial_\theta C(\theta))$ vanishes exponentially with the number of qubits, and thus, the problem size. In Appendix B we show some experiments to visualize how the proposed methodology mitigates the exponential vanishing of $\mathrm{Var}_\theta(\partial_\theta C(\theta))$ and thus, Barren Plateaus. Therefore, we demonstrate the connection between the state space reduction achieved with the constrained mixers and the trainability of the variational algorithm.

## 5 State preparation

In this part, we present a recursive algorithm for preparing a superposition state of all feasible solutions to a PBS problem defined by a tree $T$, and number of sites $N$. This algorithm generalizes the methods for preparing permutation states – for solving the TSP – proposed in [8, 9].

The state is represented by a `QuantumArray` `q_array` that consists of $M$ `QuantumVariables` representing the assigned site for each part as a one-hot encoded integer. That is, each such `QuantumVariable` consists of a register of $N$ qubits, and sites are encoded by the states where exactly one qubit is $|1\rangle$. This corresponds to an $N$-qubit $W$-state $|W_N\rangle = \frac{1}{\sqrt{N}}(|10\ldots0\rangle + |010\ldots0\rangle + |00\ldots1\rangle)$. The total number of qubits required is $M \cdot N$. In the absence of the additional constraints defined by the tree $T$, a superposition of all feasible solutions corresponds to a tensor product $\bigotimes_{r\in[M]} |W_N\rangle$. In the following, we incorporate the constraints defined by the tree $T$.

As a subroutine, we utilize the method $W(\mathtt{qv}, n)$ that takes a `QuantumVariable` of $N$ qubits and an integer $n \leq N$ as inputs, and prepares a partial $W$-state. That is, the first $n$ qubits of `qv` are in the state $|W_n\rangle$ and the last $N - n$ qubits are in state $|0\rangle$. This method can be implemented as described in Algorithm 4 with $\mathcal{O}(n)$ gates in depth $\mathcal{O}(n)$. Note that Algorithm 4 employs the $\mathrm{XX} + \mathrm{YY}$-gate which acts as a "continuous swap" depending on the angle $\phi$. This method also enables the preparation of non-uniform $W$-states depending on the angles $\phi_i$. With this, non-uniform superpositions of feasible solutions can be prepared.

Considering this, the state preparation is described in Algorithm 1. It employs Algorithm 2, 3 as subroutines. The desired superposition state of all feasible

solutions is produced recursively: Algorithm 1 initializes the `QuantumVariable` corresponding to the root of the tree $T$ in state $|W_N\rangle$, and subsequently applies Algorithm 2 to the root. Algorithm 2, when applied on a node $x \in T$, initializes the `QuantumVariables` corresponding to its predecessors $y \in \text{pred}(x)$ in partial $W$-states of decreasing size, and employs Algorithm 3 to produce an entangled state of the initialized `QuantumVariables` that satisfies the constraints (i) and (ii). Finally, Algorithm 2 is called recursively on the nodes $y \in \text{pred}(x)$.

A first inspection shows that this method can be implemented with $\mathcal{O}(M^2 \cdot N)$ gates and $\mathcal{O}(M^2 \cdot N)$ depth, with the dominant contribution resulting from enforcing the constraints (ii). A detailed analysis of this step reveals that (similarly to in [8]) stair-shaped groups of non-overlapping controlled swap gates can be executed in parallel reducing the actual circuit depth to $\mathcal{O}(M \cdot N)$. Additionally, experiments indicate that the circuit depth after processing by the *Qrisp* compiler might even scale linearly, i.e., as $\mathcal{O}(N + M)$.

**Algorithm 1:** Prepare PBS state

**Input:** A PBS tree $T$, the root *root*, the number of sites $N$, and a `QuantumArray` `q_array` in state $|0\rangle$.
**Output:** A uniform superposition of all feasible solutions.

```
/*Prepare W-state for root
  W(q_array[root], N);
/*Recursively add predecessors
  Alg. 2(T, root, N, q_array);
return q_array
```

**Algorithm 2:** Add predecessors

**Input:** A PBS tree $T$, a node $x$, the number of sites $N$, and a `QuantumArray` `q_array`.

```
/*Sorted list of predecessors of node x
  P ← pred(x);
m ← |P|;
/*Prepare partial W-states for predecessors
  for i = 0 to m − 1 do
  | W(q_array[P[i]], N − 1 − i);
/*Create entangled state of predecessors that
  satisfies constraints (ii)
  for i = m − 2 to 0 by −1 do
  | for j = i + 1 to m − 1 do
  | | Alg. 3(q_array[P[i]], q_array[P[j]], N −
  | |   2 − i);
/*Create entangled state of predecessors and x
  that satisfies constraints (i)
  for i = 0 to m − 1 do
  | Alg. 3(q_array[x], q_array[P[i]], N − 1);
/*Recursively add predecessors
  for i = 0 to m − 1 do
  | Alg. 2(T, P[i], N, q_array);
```

**Algorithm 3:** Make distinct

**Input:** `QuantumVariables qv1, qv2` in states $W_{n+1} \otimes |0\rangle$, $W_n \otimes |0\rangle$, respectively, and the index $n$.
**Output:** An entangled state $\sum_{i \neq j}^{n} |\underline{i}\rangle_1 \otimes |\underline{j}\rangle_2$, where $|\underline{i}\rangle_1$ is the state of `qv1` with exactly one 1 at position $i$.

```
for i = 0 to n − 1 do
  | with control qv1[i] do
  | | SWAP qv2[n], qv2[i];
return qv1, qv2
```

## 5.1 Reducing mixer requirements

One disadvantage of the mixer presented in Section 4 is the fact that a very large multi-controlled phase gate is required. If the PBS state is realized on $M \cdot N$ qubits, this MCP gate would require a depth of $\mathcal{O}(\log(M \cdot N))$ and a gate count of $\mathcal{O}(M \cdot N)$ by using the MCX technique presented in [11]. This can be reduced by observing an interesting property of the state preparation algorithm.

Let $\mathcal{H} = (\mathbb{C}^2)^{\otimes M \times N}$ be the Hilbert space of the $M \times N$ qubit system representing the PBS. A computational basis state $|x\rangle \in \mathcal{H}$ is *feasible* if and only if it satisfies the constraints C1, C2, C3. Let $\mathcal{H}^+$ denote the Hilbert space spanned by the feasible basis states, and $\mathcal{H}^-$ denote the Hilbert space spanned by the infeasible basis state.

Recall that a state $|x\rangle \in \mathcal{H}$ is represented by a `QuantumArray` `q_array` $= [\texttt{qv}_0, \ldots, \texttt{qv}_{M-1}]$ with $M$ `QuantumVariables`. We write $|\texttt{qv}\rangle = |\underline{i}\rangle$ where $|\underline{i}\rangle$ is the state of $N$ qubits with exactly one 1 at position $i$.

**Theorem 1.** *Let $U_{prep}\colon \mathcal{H} \to \mathcal{H}$ be the unitary corresponding to the state preparation as described in Algorithm 1. Let $|x\rangle \in \mathcal{H}$ be a computational basis state, and*

$$|y\rangle = U_{prep} |x\rangle . \tag{12}$$

*Then we have either $|y\rangle \in \mathcal{H}^+$ or $|y\rangle \in \mathcal{H}^-$. That is, the state $|y\rangle$ is either a superposition of feasible states or a superposition of infeasible states. In the latter case, this implies $\langle z| U_{prep} |x\rangle = 0$ for any feasible state $|z\rangle$.*

*Proof.* To see why this is true, let the `QuantumVariable` `qv` be an entry of the `QuantumArray` `q_array` representing $|x\rangle$ that contains at least a single `1`. Let $\nu(\texttt{qv})$ be the number of `1`s in `qv`.

The partial $W$-state initialization (i.e., Algorithm 4) flips the first qubit and then performs some "continuous swaps" to move the newly created `1` around. Applying partial $W$-state initialization to `qv` will result in a superposition of states all satisfying either of the following cases:

**Algorithm 4:** Partial $W$-state

**Input:** `QuantumVariable qv` in state $|0\rangle$, and an index $n$.
**Output:** A partial $W$-state $W_n \otimes |0\rangle$.
*/*Flip the first qubit*
  **X** $\texttt{qv}[0]$
*/*Move the 1 by "continuous swap" with angles $\phi_i$ to be determined*
  **for** $i = 1$ **to** $n-1$ **do**
    **XX+YY** $\phi_i$, $\texttt{qv}[i]$, $\texttt{qv}[0]$;
**return** `qv`

- zero 1s if $\nu(\texttt{qv}) = 1$ and $\texttt{qv}[0] = 1$,
- as least two 1s if $\nu(\texttt{qv}) > 2$, or $\nu(\texttt{qv}) = 2$ and $\texttt{qv}[0] \neq 1$, or $\nu(\texttt{qv}) = 1$ and $\texttt{qv}[0] \neq 1$,
- one 1 if $\nu(\texttt{qv}) = 2$ and $\texttt{qv}[0] = 1$.

In the first two cases, the resulting entry is therefore no longer a valid one-hot encoded integer (the condition C1 is violated). The remaining steps of the PBS state preparation algorithm consist mostly of swapping the qubits around within `qv`, implying the final value of `qv` is also not a valid one-hot encoded integer.

In the third case, let $s \in T$ be the node in the PBS tree $T$ corresponding to the `QuantumVariable qv`, i.e., $\texttt{qv} = \texttt{q_array}[s]$.

If $s$ is the root of $T$, the $W$-state initialization yields

$$|\texttt{qv}\rangle = \alpha_0 |\underline{0}\rangle + \alpha_i |\underline{i}\rangle \tag{13}$$

for some $0 < i < N$. This is a non-uniform $W_N$-state and does not yield any violation of constraints.

Otherwise, let $r \in T$ be the node such that $s \in \text{pred}(r) = \{s_0, \ldots, s_{m-1}\}$, i.e., $s = s_k$ is a predecessor of $r$. Now, let $0 < i < N-1$ be the index such that $\texttt{qv}[i] = 1$. We may assume that all `QuantumVariables` corresponding to the nodes in $r \cup \text{pred}(r)$ are either in state $|0\rangle$ or as in the third case.

If $i < N-1-k$, the partial $W$-state initialization yields

$$|\texttt{qv}\rangle = \alpha_0 |\underline{0}\rangle + \alpha_i |\underline{i}\rangle \tag{14}$$

This is a non-uniform $W_{N-1-k}$-state and does not yield any violation of constraints.

If $i \geq N-1-k$, the partial $W$-state initialization yields

$$|\texttt{qv}\rangle = |\underline{i}\rangle \tag{15}$$

We may assume that $0 \leq k \leq m-1$ is the smallest index such that this case occurs.

If $i \leq N-2$, let $l = N-2-i$ (in particular, $l < k$) and $\texttt{qv}' = \texttt{q_array}[s_l]$ be the `QuantumVariable` corresponding to node $s_l$. Then the partial $W$-state initialization yields

$$|\texttt{qv}'\rangle = \sum_{a=0}^{i} \alpha_a |\underline{a}\rangle' \tag{16}$$

Algorithm 3 applied to $(\texttt{qv}', \texttt{qv}, i)$ yields

$$\sum_{a=0}^{i} \alpha_a |\underline{a}\rangle' \otimes |\underline{i}\rangle \to \sum_{a=0}^{i} \alpha_a |\underline{a}\rangle' \otimes |\underline{a}\rangle \tag{17}$$

which violates the constraint C3 (parts $s_k$ and $s_l$ must not be at the same site). Further applications of Algorithm 3 corresponding to nodes $s_m$ for $m < l$ or $r$ will move the 1s of $\texttt{qv}'$ and $\texttt{qv}'$ to the same position, so that the constraint remains violated. If $i = N-1$, we set $\texttt{qv}' = \texttt{q_array}[r]$ and the proof is similar. In this case, the condition C2 is violated.

□

Instead of applying one large MCP gate to all qubits, we use $M$ MCP gates spanning $N$ qubits (one for each array entry). Thereby, the depth is reduced to $\mathcal{O}(\log(N))$. We have to show that this mixer still preserves the space of feasible states $\mathcal{H}^+$. The aforementioned circuit gives us the unitary

$$U_{\text{tag}}(\phi) = \sum_{x \in \{0,1\}^{M \times N}} \exp\left(i \frac{\omega(x)\phi}{M}\right) |x\rangle \langle x| \tag{18}$$

where $\omega(x)$ is the function that counts how many 0 entries $x$ contains when interpreted as a quantum array of $M$ entries. We now define the sets:

$$A = \{x \in \{0,1\}^{M \times N} \mid \omega(x) = 0\} \tag{19}$$
$$B = \{x \in \{0,1\}^{M \times N} \mid \omega(x) = M\} \tag{20}$$
$$C = \{x \in \{0,1\}^{M \times N} \mid 0 < \omega(x) < M\} \tag{21}$$

Rewriting $U_{\text{tag}}$ gives:

$$\begin{aligned} U_{\text{tag}}(\phi) &= \sum_{x \in A} |x\rangle \langle x| \\ &+ \sum_{x \in B} \exp(i\phi) |x\rangle \langle x| \\ &+ \sum_{x \in C} \exp\left(i \frac{\omega(x)\phi}{M}\right) |x\rangle \langle x| \\ &= \sum_{x \in \{0,1\}^{M \times N}} |x\rangle \langle x| \\ &+ (\exp(i\phi) - 1) |0\rangle \langle 0| \\ &+ \sum_{x \in C} \left(\exp\left(i \frac{\omega(x)\phi}{M}\right) - 1\right) |x\rangle \langle x| \\ &= \text{MCP}_0(\phi) + V(\phi) \end{aligned} \tag{22}$$

In the last step, we identify the unitary of the MCP gate with the first two terms and denote the last term by $V(\phi)$. If we now conjugate this unitary with the state preparation, we get

$$U_{\text{prep}} U_{\text{tag}}(\phi) U_{\text{prep}}^{\dagger} = U_{\text{mixer}}(\phi) + U_{\text{prep}} V(\phi) U_{\text{prep}}^{\dagger} \tag{23}$$

Let $|\psi\rangle$ be an arbitrary superposition of states from $\mathcal{H}^+$, that is,

$$|\psi\rangle = \sum_{z\in\mathcal{H}^+} \alpha_z |z\rangle \tag{24}$$

Since the PBS state is of such a form the following is especially true for it:

$$\begin{aligned} &U_{\text{prep}} V(\phi) U_{\text{prep}}^\dagger |\psi\rangle \\ &= \sum_{z\in D} \alpha_z U_{\text{prep}} V(\phi) U_{\text{prep}}^\dagger |z\rangle \\ &= \sum_{z\in D}\sum_{x\in C} \alpha_z \beta_x(\phi) U_{\text{prep}} |x\rangle \langle x| U_{\text{prep}}^\dagger |z\rangle \in \mathcal{H}^+ \end{aligned} \tag{25}$$

Here, we use that by Theorem 1, $U_{\text{prep}}|x\rangle$ lies either in $\mathcal{H}^+$ or $\mathcal{H}^-$, and in the latter case

$$\langle x| U_{\text{prep}}^\dagger |z\rangle = (\langle z| U_{\text{prep}} |x\rangle)^* = 0, \tag{26}$$

since $z \in \mathcal{H}^+$.

## 6 Implementation and Benchmarking

The described method for solving the PBS problem is implemented in *Qrisp* leveraging the existing QAOA functionalities [5] including the Trotterized Quantum Annealing (TQA) [12] parameter initialization protocol.

To summarize, the constrained QAOA exhibits the following scaling in terms of gate count and circuit depth:

- The state preparation is implemented with a few lines of code as described in Section 5. As mentioned in Section 5, it requires $\mathcal{O}(M^2 \cdot N)$ gates and circuit depth $\mathcal{O}(M \cdot N)$, and utilizes $M \cdot N$ qubits for encoding a superposition of all feasible solutions.
- The implementation of the constrained mixer, i.e., the mixer operator $U_M$, is shown below. Its complexity is dominated by the contribution from the state preparation that it employs.
- The quantum cost operator, i.e., the phase separating operator $U_P$, is generated from a `SymPy` polynomial for the classical cost function 3. This operator requires $\mathcal{O}(M^2 \cdot N)$ gates and circuit depth $\mathcal{O}(M \cdot N)$.

In total, for a QAOA with $p$ layers our algorithm utilizes $\mathcal{O}(M \cdot N)$ qubits and requires $\mathcal{O}(p \cdot M^2 \cdot N)$ gates and circuit depth $\mathcal{O}(p \cdot M \cdot N)$.

Essentially, the method is implemented in *Qrisp* as follows (here, we omit showing the implementation of straightforward classical functions):

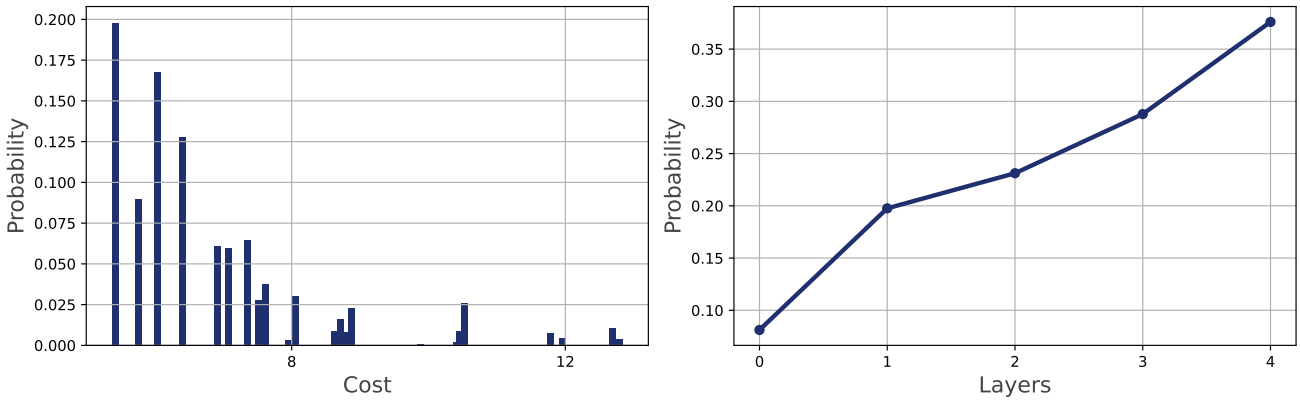
```
from qrisp import *

# Encode solutions as one-hot encoded integers
qtype = OHQInt(N)
q_array = QuantumArray(qtype = qtype, shape = (M))

"""
    Generate a SymPy polynomial and an orderd list of
     its symbols for the classical cost function for a
    dictionary of cost coefficients, number of parts M,
      number of sites N, and PBS tree.
"""
C, symbs = cost_symp(coeffs, M, N, PBS_tree)


"""
    Generate a classical cost function that can be
     evaluated on a dictionary of measurement results.
"""
cl_cost = cost_function(coeffs, PBS_tree)

# Generate quantum cost operator
def cost_op(q_array, gamma):
    app_sb_phase_polynomial(q_array, C, symbs, t=-
     gamma)

"""
    Generate the state preparation function for a
     given PBS tree and number of sites N.
    This function creates a uniform superposition
     state of all feasible solutions when applied to
     q_array.
"""
init_func = pbs_state_init(PBS_tree, N)

# Inverse state preparation function
def inv_init_func(q_array):
    with invert():
        init_func(q_array)

# Define mixer operator
def mixer_op(q_array, beta):
    with conjugate(inv_init_func)(q_array):
        for i in range(len(q_array)):
            mcp(beta, q_array[i], ctrl_state = 0)
```

With this, we have all ingredients to define the QAOA problem:

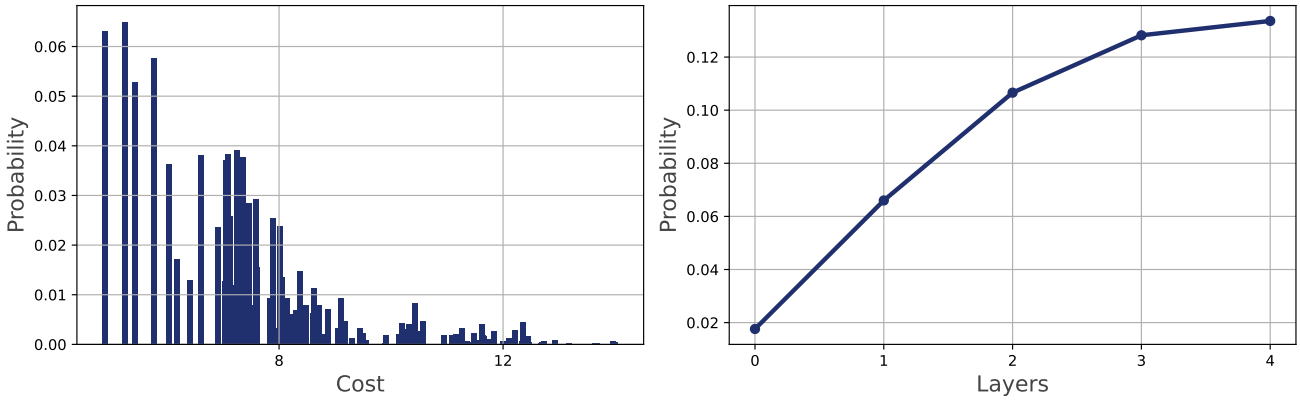
```
from qrisp.qaoa import *

# Define QAOA problem
qaoaPBS = QAOAProblem(cost_operator=cost_op,
                      mixer=mixer_op,
                      cl_cost_function=cl_cost,
                      init_type='tqa')
# Initialize QAOA in uniform superposition state of
     all feasible solutions
qaoaPBS.set_init_function(init_func)
# Number of QAOA layers
depth = 3
# Run QAOA
res = qaoaPBS.run(q_array, depth,  max_iter = 100)
```

We benchmark our method on a set of PBS instances [1]. All experiments are performed with the *Qrisp* simulator on an M2 MacBook Pro. The results are shown in Figures 2, 3.

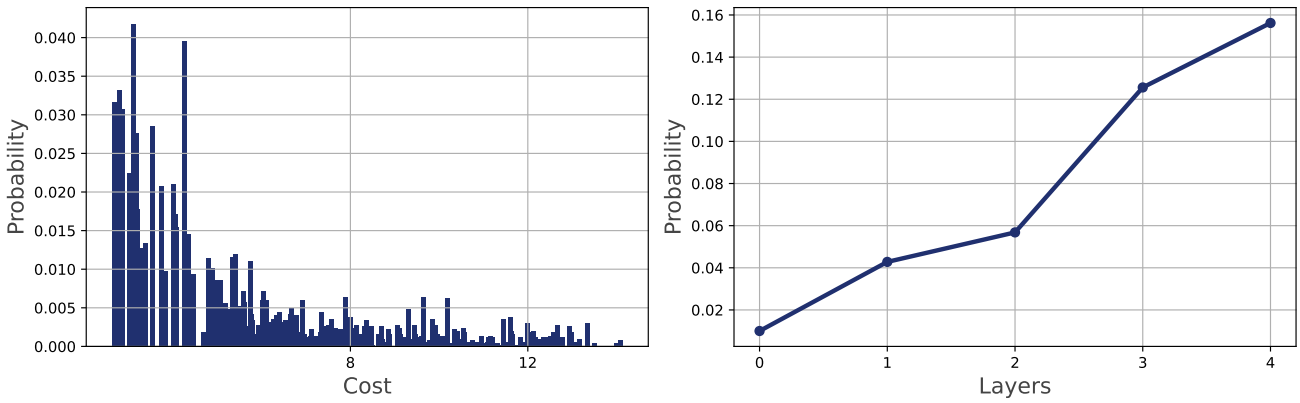
As a primary metric we choose - deviant form the frequently used approximation ratio (AR) - the **success probability** $P_\alpha$ of measuring a solution $f\colon [M] \to [N]$ such that its cost value satisfies $C(f) < (1+\alpha)\cdot C_{\min}$ , for an approximation factor $\alpha > 0$,




(a) $\phi = \{(1,0),(2,0),(3,0)\}$, $N = 4$, $|\mathcal{F}| = 24$.

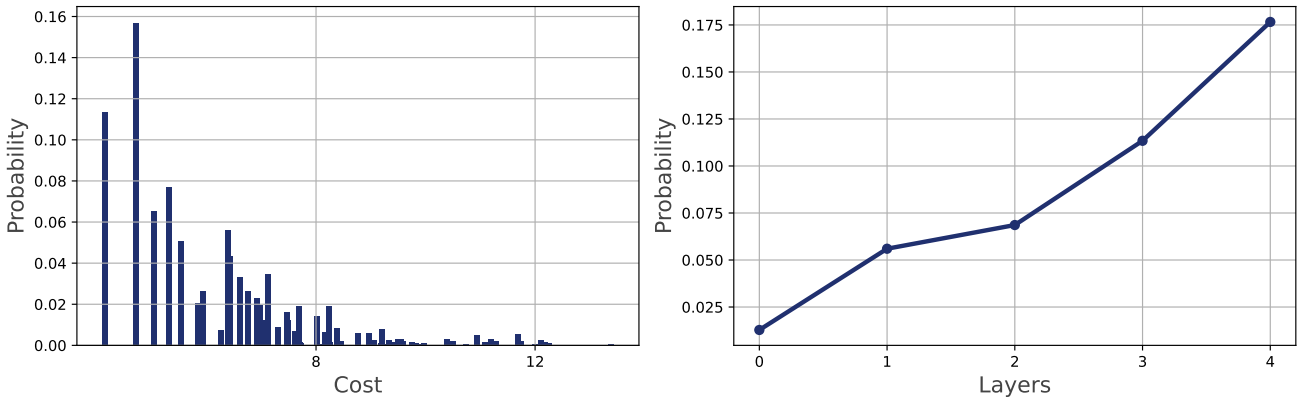



(b) $\phi = \{(1,0),(2,0),(3,0)\}$, $N = 5$, $|\mathcal{F}| = 120$.

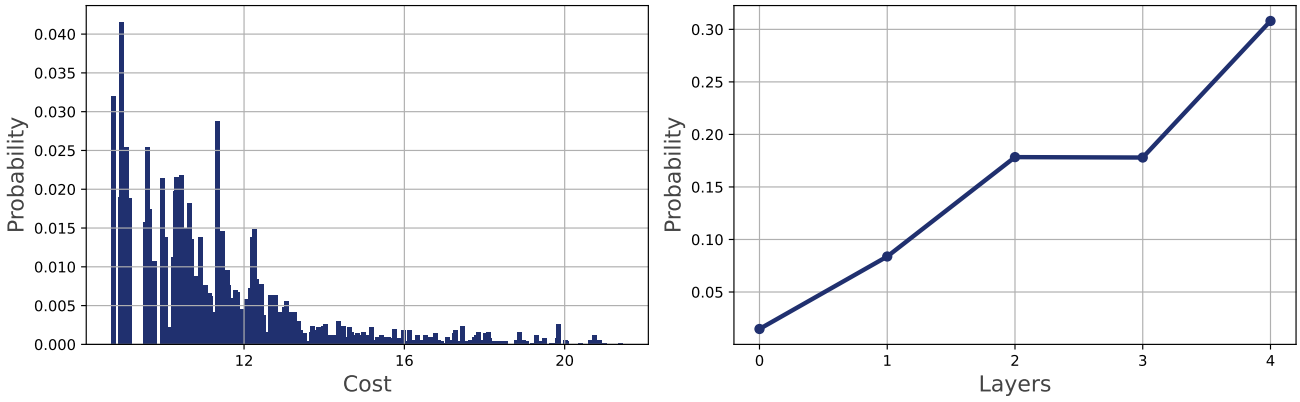



(c) $\phi = \{(1,0),(2,0),(3,0)\}$, $N = 6$, $|\mathcal{F}| = 360$.

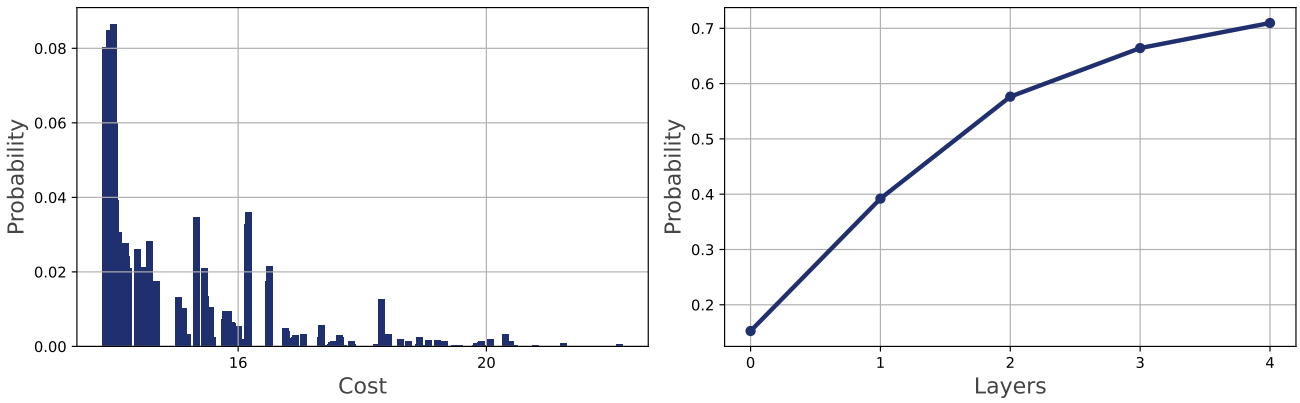
Figure 2: Left: Aggregated probabilities of measuring a feasible solution $f$ with cost $C(f)$ for QAOA with $3$ layers. Right: Success probability $P_{0.1}$ of measuring a feasible solution $f$ with cost $C(f) < 1.1 \cdot C_{\min}$ for QAOA with a varying number of layers. The case $0$ layers corresponds to a uniform superposition of all feasible solutions.




(a) $\phi = \{(1,0),(2,0),(3,1)\}$, $N = 4$, $|\mathcal{F}| = 72$.




(b) $\phi = \{(1,0),(2,0),(3,0),(4,1)\}$, $N = 5$, $|\mathcal{F}| = 480$.




(c) $\phi = \{(1,0),(2,0),(3,0),(4,1),(5,1),(6,1)\}$, $N = 4$, $|\mathcal{F}| = 144$.

Figure 3: Left: Aggregated probabilities of measuring a feasible solution $f$ with cost $C(f)$ for QAOA with $3$ layers. Right: Success probability $P_{0.1}$ of measuring a feasible solution $f$ with cost $C(f) < 1.1 \cdot C_{\min}$ for QAOA with a varying number of layers. The case $0$ layers corresponds to a uniform superposition of all feasible solutions.

where $C_{\min}$ is the cost value corresponding the optimal solution.

The results are shown in Figures 2, 3 (right) for $\alpha = 0.1$ and a varying number of layers. The findings **clearly** indicate that the QAOA routine amplifies the probabilities of (near-) optimal solutions and even scales to larger problem instances. Additionally, the aggregated probabilities of measuring a solution with a given cost are shown in Figures 2, 3 (left).

In general, the success probability reflects the ability of the QAOA routine to boost the probabilities of (near-) optimal solutions. We argue that this metric can be more meaningful than the overall approximation ratio of the resulting superposition state: Suppose that, on hard instances of an optimization problem of input size $n$ and solution space exponential in $n$, a QAOA routine achieves to boost the probability of measuring (near-) optimal solutions to a magnitude inverse polynomial in $n$. This would already be a potential advantage over classical algorithms, even if the probability of measuring a "bad" solution would be high (and therefore, the overall approximation ratio would be poor).

# 7 Conclusion and Outlook

In this work, we outline a strategy for solving the PBS problem with constrained QAOA on gate-based quantum computing devices.

Our solution is implemented in the high-level programming framework *Qrisp*. This brings the advantage of a well-arranged code base enabling rapid and seamless adoption and testing of further improvements. Potential avenues to further improve our method are the following:

- Perform some numerical estimations how fast and effective the algorithm performs for practically relevant problem scales. Similar work has been conducted by Hoefler et al. in [13].
- Develop a classification algorithm, to decide whether a PBS instance should be split using a strategy for reducing the problem size introduced below, or, alternatively, tackled with the quantum approach.
- Utilize the Quantum Minimum Finding algorithm [14] to further boost the probability of measuring a (near-) optimal solution.
- Optimize the algorithms for specific quantum devices. For example, on ion-trap systems, the global Mølmer-Sørensen gate [15] can be applied to entangle multiple qubits simultaneously. This may significantly speed up the application of the quantum cost operator $\exp(-i\gamma C)$. With *Qrisp*, this can be conveniently implemented utilizing the `GMSEnvironment`.
- Develop a method to evaluate the cost function gradient without relying on the finite differences formula. In a previous work, Wierichs et al. [16] gave general formulas for achieving this task, which might yield a custom optimizer for this problem type.

Finally, note that the strategy described in this work is not limited to the particular problem considered here. The state preparation algorithm is specific to the constraints encoded by a tree graph, but not to the optimization problem itself which is further specified by its cost function. Future work may employ the blueprint outlined in this paper to other combinatorial optimization problems with constraints.

## 7.1 Reducing the problem size

In this part, we explain how the problem structure can further be leveraged so that quantum computing and classical high-performance computing can be combined for addressing industry scale problems instances.

To be precise, we explain how a PBS instance can be decomposed, that is, how the PBS tree can be cropped into smaller instances, such that the solution of the original problem is obtained by solving smaller instances in an iterative fashion. Depending on the structure of such instances, classical and quantum algorithms may be employed. In Appendix A, we further discuss a classical dynamic programming algorithm for solving PBS problems, and thereby identify instances that are amenable to classical solvers. This is utilized to derive simple recommendations on how a PBS tree can be cropped.

Decomposing a PBS instance relies on the following key observation which is a consequence of the structure of the cost function 3: Given a tree $T$ and a node $r \in T$, let $T_r$ denote the subtree with root $r$, and $\hat{T}_r$ denote the tree obtained from $T$ by contracting the subtree $T_r$ to the node $r$. An optimal solution $f$ for the PBS instance $P = (T, \{c^x_{i,j}\}, N)$ restricts to an optimal solution $f'$ for the PBS instance $P' = (T_r, \{c^x_{i,j}\}, N)$ with initial condition $f'(r) = f(r)$. Therefore, we can crop the PBS tree at the node $r$ and find an optimal solution for the problem $P$ as follows:

- Solve the PBS instance $P'$ with the initial condition $f'(r) = i$ for all $i \in [N]$.
- Solve the PBS instance $\hat{P} = (\hat{T}_r, \{\hat{c}^x_{i,j}\}, N)$ where $\hat{c}^x_{i,j} = c^x_{i,j}$ for all $x \in \hat{T}$ and $\hat{c}^r_{i,j} = c^r_{i,j} + c^r_i$. Here, $c^r_i$ is the minimal cost for the instance $P'$ given that $f'(r) = i$.
- Extend the optimal solution $\hat{f}$ for the instance $\hat{P}$ to an optimal solution for the instance $P$ by combining it with an optimal solution for $P'$ with initial condition $f'(r) = \hat{f}(r)$.

While this method comes at the expense of solving the instance $P'$ a total number of $N$ times, these steps can be performed in parallel.

An analysis of the classical algorithm presented in Appendix A shows that a solution can be found in runtime $\mathcal{O}(M \cdot N^{m+1})$, where $m$ is the maximum degree of any node in the tree $T$. In particular, for a star tree (i.e., all nodes are immediate predecessors of the root) the runtime is in $\mathcal{O}(N^M)$, whereas for a chain the runtime is in $\mathcal{O}(M \cdot N^2)$, and for a binary tree the runtime is in $\mathcal{O}(M \cdot N^3)$. This suggests that specific scenarios, such as star trees (or, more broadly, 'bushy trees') present significant challenges for classical solutions, making quantum solutions potentially more advantageous. On the other hand, there are instances that can be solved efficiently with classical methods. This motivates the following rule for reducing the problem size: We crop the PBS tree $T$ at nodes $r$ such that the subtrees $T_r$ have a small maximum degree $m$. Additionally, further cropping of the tree and solving the resulting PBS instances with suitable classical or quantum algorithms can be applied.

Therewith, a suitable strategy for reducing the problem size can be developed based on industry relevant problems instances.

## Acknowledgment

This research was funded by the Federal Ministry for Economic Affairs and Climate Action (German: Bundesministerium für Wirtschaft und Klimaschutz), projects Qompiler (grant agreement no: 01MQ22005A) and EniQmA (grant agreement no: 01MQ22007A), and by the European Union, project OASEES (HORIZON-CL4-2022, grant agreement no 101092702). The authors are responsible for the content of this publication.

## Data availability

All data generated during this study are included in this published article.

## Code availability

*Qrisp* is an open-source Python framework for high-level programming of quantum computers. The source code is available in https://github.com/eclipse-qrisp/Qrisp. The underlying code and datasets for this study are available in https://github.com/renezander90/PBS-QAOA.

## References

[1] Airbus and BMW Group. "Quantum-powered logistics: Towards an efficient and sustainable supply chain". https://qcc.thequantuminsider.com/wp-content/uploads/2023/12/QCChallenge_QOPT_AirbusBMWGroup_v1.1.pdf. Accessed: 30.04.2024.

[2] Tjalling C Koopmans and Martin Beckmann. "Assignment problems and the location of economic activities". Econometrica: Journal of the Econometric SocietyPages 53–76 (1957). url: https://elischolar.library.yale.edu/cowles-discussion-paper-series/221/.

[3] Raphael Seidel, Sebastian Bock, René Zander, Matic Petrič, Niklas Steinmann, Nikolay Tcholtchev, and Manfred Hauswirth. "Qrisp: A framework for compilable high-level programming of gate-based quantum computers". To appear (2024).

[4] Raphael Seidel, René Zander, Matic Petrič, Niklas Steinmann, David Q Liu, Nikolay Tcholtchev, and Manfred Hauswirth. "Quantum backtracking in Qrisp applied to Sudoku problems" (2024). arXiv:2402.10060.

[5] Eneko Osaba, Matic Petrič, Izaskun Oregi, Raphael Seidel, Alejandra Ruiz, and Michail-Alexandros Kourtis. "Eclipse Qrisp QAOA: description and preliminary comparison with Qiskit counterparts" (2024). arXiv:2405.20173.

[6] Edward Farhi, Jeffrey Goldstone, and Sam Gutmann. "A quantum approximate optimization algorithm" (2014). arXiv:1411.4028.

[7] Stuart Hadfield, Zhihui Wang, Bryan O'gorman, Eleanor G Rieffel, Davide Venturelli, and Rupak Biswas. "From the quantum approximate optimization algorithm to a quantum alternating operator ansatz". Algorithms **12**, 34 (2019).

[8] Andreas Bärtschi and Stephan Eidenbenz. "Grover mixers for QAOA: Shifting complexity from mixer design to state preparation". In 2020 IEEE International Conference on Quantum Computing and Engineering (QCE). Pages 72–82. (2020).

[9] Atsushi Matsuo, Yudai Suzuki, Ikko Hamamura, and Shigeru Yamashita. "Enhancing VQE convergence for optimization problems with problem-specific parameterized quantum circuits". IEICE TRANSACTIONS on Information and Systems **106**, 1772–1782 (2023).

[10] Jarrod R. McClean, Sergio Boixo, Vadim N. Smelyanskiy, Ryan Babbush, and Hartmut Neven. "Barren plateaus in quantum neural network training landscapes". Nature Communications**9** (2018).

[11] Stefan Balauca and Andreea Arusoaie. "Efficient constructions for simulating multi controlled quantum gates". In Computational Science – ICCS 2022. Pages 179–194. Springer International Publishing (2022).

[12] Stefan H Sack and Maksym Serbyn. "Quantum annealing initialization of the quantum approximate optimization algorithm". Quantum **5**, 491 (2021).

[13] Torsten Hoefler, Thomas Häner, and Matthias Troyer. "Disentangling hype from practicality: On realistically achieving quantum advantage". Communications of the ACM **66**, 82–87 (2023).

[14] Joran van Apeldoorn, András Gilyén, Sander Gribling, and Ronald de Wolf. "Quantum SDP-Solvers: Better upper and lower bounds". Quantum **4**, 230 (2020).

[15] Klaus Mølmer and Anders Sørensen. "Multiparticle entanglement of hot trapped ions". Physical Review Letters **82**, 1835–1838 (1999).

[16] David Wierichs, Josh Izaac, Cody Wang, and Cedric Yen-Yu Lin. "General parameter-shift rules for quantum gradients". Quantum **6**, 677 (2022).

[17] Martin Larocca, Piotr Czarnik, Kunal Sharma, Gopikrishnan Muraleedharan, Patrick J. Coles, and M. Cerezo. "Diagnosing barren plateaus with tools from quantum optimal control". Quantum **6**, 824 (2022).

# A Classical optimization

A classical dynamic programming method for solving the PBS problem is described in Algorithm 5. This method is amenable to a complexity theoretic analysis of its runtime and thereby facilitates distinguishing classically solvable instances from intractable ones.

Recall that $m = \max_{x \in T} |\mathrm{pred}(x)|$ is the maximal number of predecessors of any node in the tree $T$. Then Algorithm 5 finds the optimal solution in runtime $\mathcal{O}(M \cdot N^{m+1})$: For each node $x \in [M]$, and each assignment $f(x) = i \in [N]$, it explores all feasible assignments $f(y) = j \in [N]$ for $y \in \mathrm{pred}(x)$.

**Algorithm 5:** Classical PBS solution

**Input:** A PBS instance $P = (T, \{c^x_{i,j}\}, N)$.
**Output:** The optimal solution $f\colon [M] \to [N]$.

```
/*Dictionary to store partial solutions
 J ← ∅
/*Let the height h(x) of a node x in a tree be
 the number of edges on the longest path from
 the node down to a leaf. A leaf has height of 0.
 The height of a tree h(T) is the height of its
 root. Let T_h be the set of nodes of height h.

for x ∈ T_0 do
    for i = 1 to N do
        c^i_x ← 0;
for h = 1 to h(T) do
    for x ∈ T_h do
        for i = 1 to N do
            c^x_i ← min_{j(y)} { Σ_{y∈pred(x)} (c^y_j + c^y_{i,j}) };
            /*j*(y) the optimal feasible
              assignment of sites j ∈ [N] to parts
              y ∈ pred(x) given that x is at site i.
              J[x, i] ← j*(y);
/*Find optimal cost value and assignment for
 the root.
 C ← min_{i∈[N]} {c^i_r};
Recover optimal solution f from J
```

# B Variance of Cost Function's gradients

As aforementioned in Section 4, the proposed constrained mixer solution lightens the burden of vanishing gradients. In Figure 4 we show that (increasing the problem size according to the experimental setup in Table 1) the decaying of the variance is less impactful compared to other variational algorithms. Specifically, in the case of the most known variational algorithms affected by Barren Plateaus, the variance becomes significantly smaller by several orders of magnitude, even with circuits with a much smaller number of qubits compared to our experiments [17].

| $N_q$ | $N$ | $\phi$ |
|---|---|---|
| 12 | 3 | $\{(1,0),(2,0),(3,1)\}$ |
| 16 | 4 | $\{(1,0),(2,0),(3,1))\}$ |
| 20 | 4 | $\{(1,0),(2,0),(3,1),(4,2)\}$ |
| 24 | 4 | $\{(1,0),(2,0),(3,1),(4,2),(5,4)\}$ |

Table 1: Setup for the experiments shown in Figure 4. $N_q$ is the number of qubits and $N$ the number of sites.

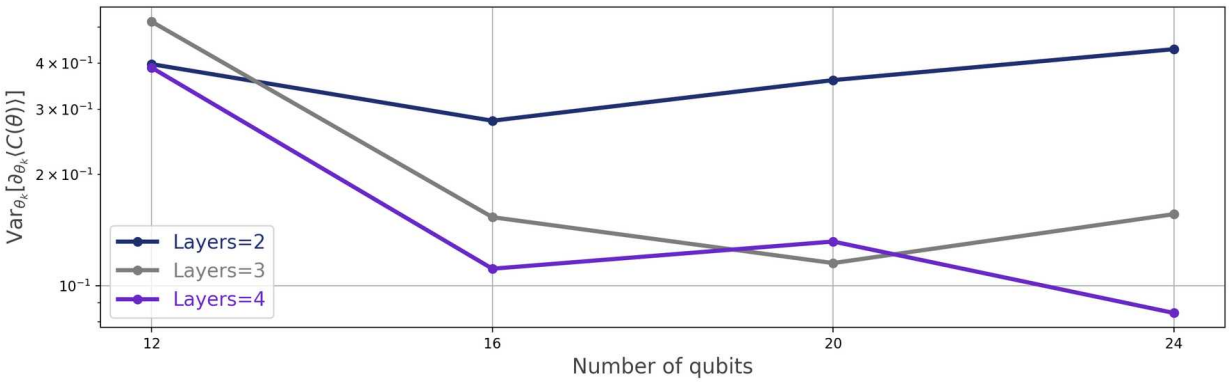



Figure 4: Logarithmical plot visualizing the variance of the cost function's gradients of the proposed solution's Ansatz. The experiments are done increasing the problem sizes as reported in Table 1. The gradient of the cost function is computed with the finite difference method w.r.t. the last parameter of the Ansatz, and the variance is calculated with 100 samples.